IMPLEMENTACIÓN DE NGINX EN KUBERNETES SOBRE APACHE CLOUDSTACK

Dennis.A. Andrade Guaman, E.Henry. Carvajal Pabon, Edison .J. Chiluiza Gualpa, Erica .L. Mendoza Albán, Miguel .A. Morocho Pilataxi, Caterine.R. Muzo Samueza, David.J. Saraguro Rendón, Tiffani .N. Torres Díaz

Departamento de Ciencias de la Computación, Universidad de las Fuerzas Armadas-ESPE

Sangolquí, Ecuador

daandrade11@espe.edu.ec
ehcarvajal@espe.edu.ec
ejchiluiza@espe.edu.ec
elmendoza2@espe.edu.ec
mamorocho7@espe.edu.ec
crmuzo@espe.edu.ec
djsaraguro@espe.edu.ec
tntorres@espe.edu.ec

1. Descripción del escenario

En este laboratorio se realizó el despliegue de un servidor web Nginx utilizando Kubernetes dentro de una infraestructura de Apache CloudStack, para esto se utilizó Ubuntu mediante WSL2 con virtualización KVM, además se preparó un clúster con dos nodos, uno de control y otro de trabajo, donde se ejecutó la aplicación mediante un archivo YAML que contiene un Deployment y un Service.

Durante la práctica se comprobó el funcionamiento del servidor desde el navegador, después se aumentó la cantidad de réplicas y finalmente se eliminaron los recursos utilizados, esto permitió conocer cómo Kubernetes administra las aplicaciones dentro de una infraestructura virtualizada.

2. Prerrequisitos

Componente	Condición necesaria
Entorno base	Windows con WSL2 y una distribución Ubuntu dedicada al laboratorio, denominada CloudStack-D2.
Virtualización	Dispositivo /dev/kvm disponible y comprobación satisfactoria de creación de una máquina virtual y una CPU virtual.
Recursos	Memoria, CPU y almacenamiento suficientes para ejecutar CloudStack, sus máquinas de sistema y los nodos Kubernetes. En el entorno utilizado se configuró un límite de 10 GB de memoria compartida para WSL2.
Infraestructura	Apache CloudStack instalado, con zona, host KVM, red y almacenamiento configurados y operativos.
Clúster Kubernetes	Un nodo de control y un nodo de trabajo en estado Ready. Las capturas muestran la versión v1.35.4.
Administración	Herramienta kubectl configurada con acceso al clúster y permisos para crear, consultar, escalar y eliminar los recursos.
Conectividad	Acceso al registro de imágenes para descargar Nginx y comunicación entre los componentes del clúster.
Publicación web	Service NodePort en el puerto 30080 y reglas de red y reenvíos necesarios para acceder desde Windows mediante http://localhost:18080/.
Archivo de despliegue	Manifiesto nginx-web.yaml con las definiciones del Deployment y del Service.

Para realizar el laboratorio se utilizó WSL2 con la virtualización anidada habilitada, acceso a KVM y una configuración de 10 GB de memoria para WSL, estos recursos permitieron preparar el entorno necesario para CloudStack y Kubernetes, aunque los 10 GB corresponden a la configuración utilizada en esta práctica y no representan un requisito mínimo oficial.

3. Imagen del contenedor

Para ejecutar el servidor web se utilizó la imagen nginx:stable-alpine, que corresponde a una versión de Nginx basada en Alpine Linux, esta imagen se configuró dentro del archivo YAML para que Kubernetes pudiera crear los contenedores necesarios, además hay que considerar que la etiqueta stable-alpine puede actualizarse por lo que para utilizar exactamente la misma imagen en otro despliegue sería necesario registrar su digest.

Propiedad	Valor utilizado
Imagen	nginx:stable-alpine
Registro	Docker Hub, repositorio library/nginx
Función	Proporcionar el servidor web de la aplicación.
Puerto del contenedor	80/TCP
Réplicas	Una inicialmente; dos durante la prueba de escalado.

Propiedad	Valor utilizado
Recursos solicitados por contenedor	50m de CPU y 32Mi de memoria.
Límites por contenedor	250m de CPU y 128Mi de memoria.
Comprobación de disponibilidad	Petición HTTP a la ruta / en el puerto 80.

```
containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
      - containerPort: 80
```

01. Sección 1: Preparación y verificación del entorno

En esta sección se preparó el entorno de Ubuntu mediante WSL2 y se comprobó el funcionamiento de KVM, además se revisaron los recursos necesarios para continuar con la instalación de CloudStack.

Paso 1. Preparación de Ubuntu en WSL2

Para comenzar se creó una distribución independiente llamada CloudStack-D2 utilizando Ubuntu 24.04, con la finalidad de tener un entorno dedicado al laboratorio sin modificar la distribución Ubuntu que ya estaba instalada, después se utilizó el comando wsl --list --verbose para comprobar que ambas distribuciones estuvieran registradas y utilizando WSL2, esto permitió trabajar con entornos separados dentro del mismo equipo.

```
wsl --list --verbose
```



Fig. 1 Distribuciones Ubuntu y CloudStack-D2 registradas en WSL2.

Paso 2. Comprobación del acceso a KVM

Antes de instalar CloudStack se realizó una prueba de virtualización desde Ubuntu utilizando un script de Python, como resultado se logró crear y cerrar una máquina virtual vacía junto con una CPU virtual, esto permitió comprobar que KVM estaba disponible para realizar operaciones de virtualización, aunque todavía no significa que CloudStack estuviera instalado ni que la misma prueba se hubiera ejecutado dentro de CloudStack-D2.

```
wsl.exe -d Ubuntu -u root -- python3
"/home/denis/F3/pasos 3/evidencias/probar-kvm.py"
```

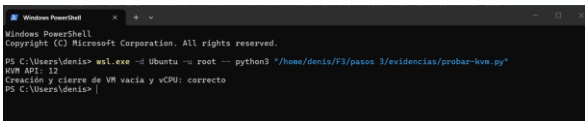


Fig. 2 Verificación funcional de KVM mediante la creación y cierre de una máquina virtual vacía y una CPU virtual.

Paso 3. Configuración de memoria y virtualización

Después se configuró WSL2 con un límite de 10 GB de memoria y se habilitó la virtualización anidada mediante el archivo .wslconfig, una vez realizados los cambios se reinició WSL utilizando wsl --shutdown para aplicar la configuración.

También se utilizaron comandos como systemctl, free -h y ls -l /dev/kvm desde CloudStack-D2 para comprobar el funcionamiento de systemd, que no existieran servicios fallidos y que estuvieran disponibles la memoria y el dispositivo KVM, esto permitió verificar las condiciones del entorno antes de continuar con la instalación.

```
# Archivo: C:\Users\denis\.wslconfig
[wsl2]
memory=10GB
nestedVirtualization=true

# Aplicar la configuración en PowerShell
wsl --shutdown
```

```
wsl.exe -d CloudStack-D2 -u root -- bash -lc
"systemctl is-system-running; systemctl --failed --
no-pager; free -h; ls -l /dev/kvm"
```



Fig. 3 Verificación del entorno CloudStack-D2 y disponibilidad del dispositivo de virtualización KVM.

02. Sección 2: Instalación de CloudStack y clúster Kubernetes

En esta sección se preparó Apache CloudStack con los recursos necesarios para utilizar Kubernetes, además se comprobó el funcionamiento del clúster donde posteriormente se ejecutaría el servidor Nginx.

Paso 1. Instalación de Apache CloudStack

Según los registros del laboratorio se instaló Apache CloudStack 4.22.1.1 junto con MySQL dentro de CloudStack-D2, además se configuraron KVM, una red dedicada y almacenamiento NFS para preparar la infraestructura, después se ingresó desde el navegador de Windows a http://localhost:8080/client/ donde se pudo observar la consola administrativa, esto permitió comprobar que la interfaz de CloudStack se encontraba disponible.

```
http://localhost:8080/client/
```

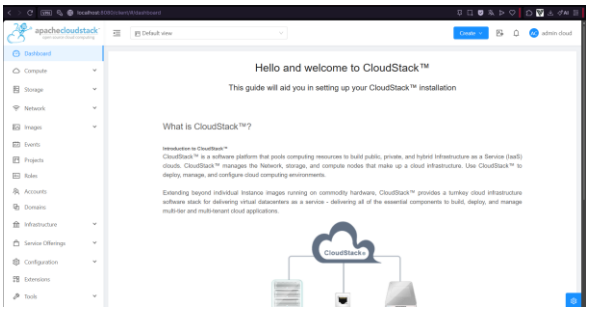


Fig. 4 Pantalla inicial de la consola de Apache CloudStack.

Paso 2. Configuración de la zona y almacenamiento

Después se configuró una zona llamada Zona-D2 con un host KVM y almacenamiento NFS primario y secundario, además según el registro las máquinas de sistema s-1-VM y v-2-VM se encontraban en estado Running con sus agentes Up, esto indica que los componentes estaban funcionando, aunque esta comprobación se encuentra registrada en las notas y no dispone de una captura específica.

Paso 3. Comprobación del clúster Kubernetes

Una vez preparado CloudStack se verificó el funcionamiento del clúster Kubernetes utilizando el comando kubectl get nodes, como resultado se identificaron dos nodos, uno de control y otro de trabajo, ambos en estado Ready utilizando Kubernetes v1.35.4, además el registro indicó que existían 14 pods internos en ejecución, esto permitió comprobar que el clúster estaba preparado para continuar con el despliegue de Nginx.

```
wsl.exe -d CloudStack-D2 -u root -- kubectl get
nodes
```

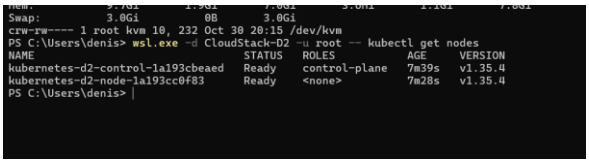


Fig. 5 Nodos del clúster Kubernetes en estado Ready.

03. Sección 3: Despliegue y acceso al servidor Nginx

En esta sección se preparó el archivo YAML para configurar el servidor Nginx dentro de Kubernetes, después se crearon los recursos necesarios y se comprobó que la página pudiera abrirse desde el navegador de Windows.

Paso 1. Preparación del archivo nginx-web.yaml

Para comenzar se creó el archivo nginx-web.yaml donde se configuraron un Deployment y un Service para ejecutar Nginx, el Deployment utiliza la imagen nginx:stable-alpine con una réplica inicial mientras que el Service permite publicar el servidor mediante NodePort 30080 conectado con el puerto 80 del contenedor.

Además se configuraron solicitudes de 50m de CPU y 32 MiB de memoria con límites de 250m de CPU y 128 MiB, también se agregó una comprobación HTTP para verificar que Nginx estuviera preparado para recibir solicitudes, antes de aplicar el archivo se utilizó kubectl apply --dry-run=server para validar el manifiesto y comprobar que Kubernetes aceptara su configuración.

```
wsl.exe -d CloudStack-D2 -u root -- cat
/root/cloudstack-d2/nginx-web.yaml
```

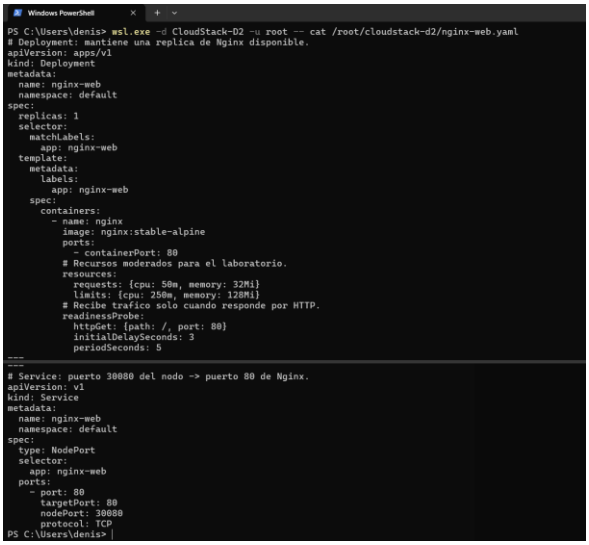


Fig. 6 Manifiesto nginx-web.yaml: Deployment y continuación del Service (capturas 8 y 8.1).

Código completo del manifiesto nginx-web.yaml

Deployment: definición del servidor web

```
# Deployment: mantiene una replica de Nginx
disponible.
apiVersion: apps/v1    # API que define los
Deployments
kind: Deployment    # Controlador que mantiene los
pods
metadata:    # Información identificadora
  name: nginx-web    # Nombre del Deployment
  namespace: default    # Espacio de nombres usado
spec:    # Configuración del Deployment
  replicas: 1    # Cantidad inicial de réplicas
  selector:    # Reconoce los pods administrados
    matchLabels:    # Aplica el filtro por etiquetas
      app: nginx-web    # Etiqueta exigida al pod
  template:    # Define la plantilla del pod
    metadata:    # Información del pod
      labels:    # Conjunto de etiquetas del pod
        app: nginx-web    # Etiqueta que enlaza los
recursos
    spec:    # Especificación interna del pod
      containers:    # Contenedores que ejecutará el
pod
        - name: nginx    # Nombre asignado al
contenedor
          image: nginx:stable-alpine    # Imagen de
Nginx en Docker Hub
          ports:    # Puertos que declara el
contenedor
            - containerPort: 80    # Puerto HTTP del
contenedor
          # Recursos moderados para el laboratorio.
          resources:    # Configura CPU y memoria
            requests: {cpu: 50m, memory: 32Mi} #
Recursos que solicita el contenedor
            limits: {cpu: 250m, memory: 128Mi} #
Valores máximos permitidos
          # Recibe trafico solo cuando responde por
HTTP.
          readinessProbe:    # Prueba de
disponibilidad del pod
            httpGet: {path: /, port: 80}    #
Comprueba HTTP en /, puerto 80
            initialDelaySeconds: 3    # Espera
inicial de 3 segundos
            periodSeconds: 5    # Ejecuta la prueba
cada 5 segundos
---    # Separador entre recursos YAML
```

Service: publicación mediante NodePort

```
# Service: puerto 30080 del nodo -> puerto 80 de
Nginx.
apiVersion: v1    # API utilizada para Service
kind: Service    # Recurso que publica los pods
metadata:    # Datos identificadores del Service
  name: nginx-web    # Nombre del Service
  namespace: default    # Espacio de nombres del
Service
spec:    # Configuración de publicación
  type: NodePort    # Expone un puerto en cada nodo
  selector:    # Selecciona los pods de destino
    app: nginx-web    # Etiqueta del pod seleccionado
  ports:    # Lista de puertos del Service
    - port: 80    # Puerto interno del Service
      targetPort: 80    # Puerto de destino en el
contenedor
      nodePort: 30080    # Puerto expuesto en los
nodos
      protocol: TCP    # Protocolo de transporte
```

Explicación de los campos del manifiesto

Dentro del archivo YAML se utilizaron diferentes campos para configurar los recursos de Kubernetes, primero apiVersion y kind permiten identificar la versión de la API y el tipo de recurso que se va a crear, mientras que metadata establece información como el nombre nginx-web y el espacio de nombres default, esto permite reconocer los recursos dentro del clúster.

El campo replicas establece la cantidad de instancias que debe mantener Kubernetes, en este caso inicialmente se configuró una, además se utilizaron selector, matchLabels y template para relacionar el Deployment con sus pods mediante la etiqueta app: nginx-web, lo que permite identificar qué contenedores pertenecen a la aplicación.

También se configuró el contenedor con el nombre nginx y la imagen nginx:stable-alpine, utilizando containerPort: 80 para recibir las solicitudes HTTP, además en resources se establecieron solicitudes de 50m de CPU y 32 MiB de memoria con límites de 250m y 128 MiB, esto permite controlar los recursos solicitados y los máximos permitidos para el contenedor.

Por otra parte, readinessProbe permite comprobar si Nginx está preparado para recibir tráfico mediante una solicitud HTTP a la ruta principal, con una espera inicial de tres segundos y comprobaciones cada cinco segundos, esto ayuda a que Kubernetes identifique cuando el servidor se encuentra disponible.

Finalmente el Service utiliza type: NodePort para publicar la aplicación mediante los nodos del clúster, mientras que selector permite identificar los pods correspondientes, además port y targetPort establecen la comunicación mediante el puerto 80 y nodePort configura el puerto 30080 utilizando el protocolo TCP, con esto se prepara el acceso al servidor desde la red correspondiente.

Paso 2. Creación del Deployment y Service

Después de validar el archivo YAML se ejecutó el comando kubectl apply -f /root/cloudstack-d2/nginx-web.yaml para crear los recursos de Nginx dentro del clúster, como resultado la terminal mostró que tanto el Deployment como el Service fueron creados correctamente, esto permitió comprobar que Kubernetes aceptó la configuración y comenzó a preparar el servidor para su funcionamiento.

```
kubectl apply -f /root/cloudstack-d2/nginx-
web.yaml
```

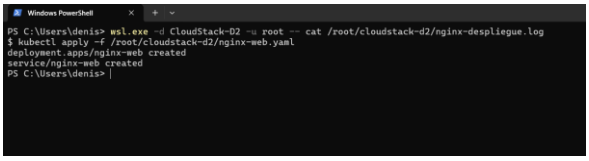


Fig. 7 Creación correcta del Deployment y del Service nginx-web.

Paso 3. Verificación de los recursos de Kubernetes

Una vez creados los recursos se comprobó su estado utilizando comandos como kubectl get deployments, kubectl get pods y kubectl get services, obteniendo una réplica disponible en estado 1/1 y un pod Running sin reinicios, además ambos nodos permanecieron en estado Ready.

También se verificó que el Service utilizara NodePort 30080 mientras que desde Windows se configuró el puerto local 18080, esto permitió preparar el acceso a la aplicación desde el navegador sin necesidad de ingresar directamente al clúster.

```
kubectl get deployments
kubectl get pods -o wide
kubectl get services
kubectl get nodes -o wide
```

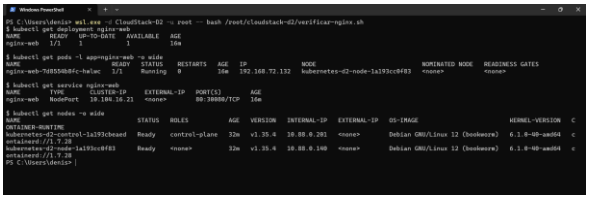


Fig. 8 Verificación de los recursos y nodos activos del clúster.

Paso 4. Comprobación de la página Nginx

Para comprobar el funcionamiento del servidor se abrió el navegador de Windows ingresando la dirección http://localhost:18080/, donde se pudo observar el mensaje Welcome to nginx!, además según el registro la respuesta HTTP fue 200 OK lo que indica que el servidor atendió correctamente la solicitud.

Esto fue posible mediante un reenvío desde el puerto local 18080 hacia el NodePort 30080 de Kubernetes, por lo que es importante diferenciar ambos puertos ya que uno permite acceder desde Windows mientras que el otro corresponde al servicio publicado dentro del clúster.

```
http://localhost:18080/
```

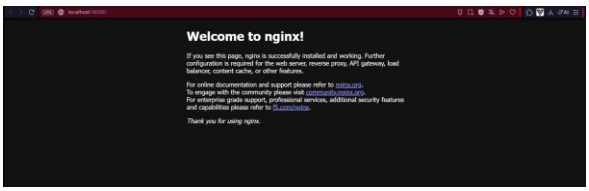


Fig. 9 Página de bienvenida de Nginx accesible desde Windows.

04. Sección 4: Escalado y eliminación de los recursos

Finalmente se comprobó cómo Kubernetes permite aumentar la cantidad de réplicas de una aplicación y eliminar los recursos utilizados sin necesidad de borrar toda la infraestructura de CloudStack.

Paso 1. Escalado del servidor Nginx

Después de comprobar que Nginx funcionaba correctamente se aumentó la cantidad de réplicas de una a dos utilizando kubectl scale deployment nginx-web --replicas=2, posteriormente se revisó el estado del Deployment y los pods, como resultado se obtuvieron dos réplicas disponibles en estado 2/2 y dos pods Running sin reinicios, esto demuestra que Kubernetes pudo crear otra instancia del servidor manteniendo ambas disponibles.

```
kubectl scale deployment nginx-web --replicas=2
kubectl get deployment nginx-web
kubectl get pods -l app=nginx-web
```

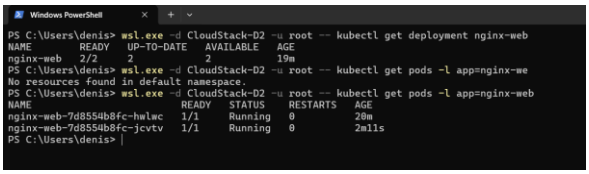


Fig. 10 Escalado correcto de Nginx a dos réplicas disponibles.

Paso 2. Eliminación de los recursos

Una vez terminadas las pruebas se eliminaron el Deployment y el Service mediante kubectl delete -f /root/cloudstack-d2/nginx-web.yaml, después se comprobó que ya no existieran pods relacionados con nginx-web, además se retiró la regla del puerto 30080 y se deshabilitó el proxy local utilizado para acceder desde Windows.

Con esto se confirmó que los recursos de Nginx fueron eliminados y que la página dejó de estar disponible desde localhost:18080, sin embargo CloudStack, el clúster Kubernetes y los archivos de evidencia permanecieron conservados, lo que permite mantener el entorno preparado sin repetir toda la instalación.

```
kubectl delete -f /root/cloudstack-d2/nginx-web.yaml
kubectl get deployment nginx-web
kubectl get service nginx-web
kubectl get pods -l app=nginx-web
```

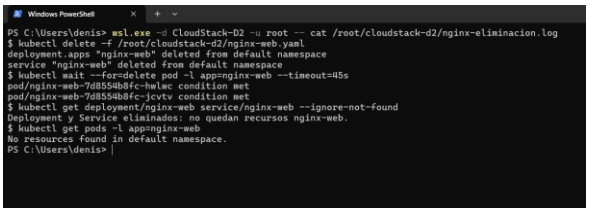


Fig. 11 Eliminación del Deployment, Service y pods de la práctica.